

Part 5: Basic Tree Properties

Your first ten recursive tree functions

One template, many problems

Here is the template from Part 1 again. Every function in this part is a filled-in copy of it.

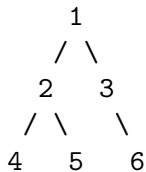
```
ReturnType solve(TreeNode* root) {  
    if (root == NULL) return answerForEmptyTree;  
  
    ReturnType L = solve(root->left);  
    ReturnType R = solve(root->right);  
  
    return combine(L, R, root->val);  
}
```

Writing a tree function is therefore a two question process:

1. **What is the answer for an empty tree?** That is your base case.
2. **If a genie handed me the correct answer for the left subtree and the correct answer for the right subtree, how would I build my answer?** That is your combine step.

Notice what you are *not* doing: you are not tracing the recursion in your head. Trust the genie. If the base case is right and the combine step is right, the recursion is right. This is the single biggest mindset shift for a beginner, and it takes a few problems before it feels natural.

Working tree for this part:



Maximum depth, also called height

LeetCode 104. Counting nodes, so a single node has depth 1.

```
int maxDepth(TreeNode* root) {  
    if (root == NULL) return 0;
```

```

int L = maxDepth(root->left);
int R = maxDepth(root->right);
return 1 + max(L, R);
}

```

Base case: an empty tree has no levels, so 0. Combine: my depth is the deeper of my two children, plus one for me.

Dry run

```

maxDepth(4) = 1 + max(0, 0) = 1
maxDepth(5) = 1 + max(0, 0) = 1
maxDepth(2) = 1 + max(1, 1) = 2
maxDepth(6) = 1 + max(0, 0) = 1
maxDepth(3) = 1 + max(0, 1) = 2
maxDepth(1) = 1 + max(2, 2) = 3

```

Answer 3. Note the order in which those lines were completed: 4, 5, 2, 6, 3, 1. That is the postorder. Every bottom-up computation happens in postorder, whether you call it that or not.

If you want the height measured in **edges** instead, the only change is the base case: return -1 for NULL. Then a single node gives $1 + \max(-1, -1) = 0$. Nothing else moves.

Time $O(N)$, space $O(H)$.

Minimum depth

LeetCode 111. The number of nodes on the shortest path from the root down to a **leaf**.

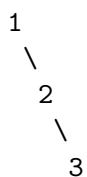
The obvious answer is wrong:

```

// WRONG
int minDepth(TreeNode* root) {
    if (root == NULL) return 0;
    return 1 + min(minDepth(root->left), minDepth(root->right));
}

```

Try it on



The left subtree of 1 is empty and returns 0, so the answer comes out as 1. But node 1 is not a leaf. The true answer is 3.

The problem is that “empty subtree” and “path ends here” are different things. A missing child does not create a leaf.

```
int minDepth(TreeNode* root) {  
    if (root == NULL) return 0;  
    if (root->left == NULL) return 1 + minDepth(root->right);  
    if (root->right == NULL) return 1 + minDepth(root->left);  
    return 1 + min(minDepth(root->left), minDepth(root->right));  
}
```

Read the middle two lines as: if I have only one child, the shortest path to a leaf must go through that child, there is no choice.

On our tree: node 3 has only a right child, so $\text{minDepth}(3) = 1 + \text{minDepth}(6) = 2$. Node 2 has both, so $\text{minDepth}(2) = 1 + \min(1, 1) = 2$. Root: $1 + \min(2, 2) = 3$.

Time $O(N)$, space $O(H)$. The BFS version from Part 3 has the same worst case but stops early in practice.

Count nodes

```
int countNodes(TreeNode* root) {  
    if (root == NULL) return 0;  
    return 1 + countNodes(root->left) + countNodes(root->right);  
}
```

Empty tree has 0 nodes. My count is myself plus both subtrees. On our tree: 6.

Time $O(N)$, space $O(H)$.

(LeetCode 222 asks this for a *complete* tree and wants better than $O(N)$. That uses the special shape and is not needed here.)

Count leaf nodes

A leaf has **both** children NULL.

```
int countLeaves(TreeNode* root) {  
    if (root == NULL) return 0;  
    if (root->left == NULL && root->right == NULL) return 1;  
    return countLeaves(root->left) + countLeaves(root->right);  
}
```

Two base cases here. The NULL case protects against crashes, the leaf case is where the counting actually happens. On our tree: leaves are 4, 5, 6, so the answer is 3.

A very common wrong version writes `if (root->left == NULL || root->right == NULL) return 1;`. That would count node 3, which has one child, as a leaf. Use `&&`.

Sum of all nodes

```
int sumOfNodes(TreeNode* root) {
    if (root == NULL) return 0;
    return root->val + sumOfNodes(root->left) + sumOfNodes(root->right);
}
```

Our tree: $1 + 2 + 3 + 4 + 5 + 6 = 21$.

Compare this with `countNodes`. The two functions are identical except that one adds `root->val` and the other adds 1. Recognise that and you start to see the template rather than the individual problems.

Maximum value in the tree

```
int maxValue(TreeNode* root) {
    if (root == NULL) return INT_MIN;
    return max(root->val, max(maxValue(root->left), maxValue(root->right)));
}
```

The base case deserves a moment. What is the maximum of an empty set? There is no such number, so we return the most negative value possible, which loses every `max` comparison and therefore behaves like “no contribution”. This trick, returning an identity element for the empty case, appears constantly:

Operation	Identity to return for NULL
sum	0
count	0
max	INT_MIN
min	INT_MAX
logical AND (“all nodes satisfy”)	true
logical OR (“some node satisfies”)	false
height in nodes	0
height in edges	-1

Memorise this table. It answers the base case question for most problems immediately.

Search for a value

```
bool search(TreeNode* root, int target) {
    if (root == NULL) return false;
    if (root->val == target) return true;
    return search(root->left, target) || search(root->right, target);
}
```

In a general binary tree there is no ordering, so you may have to look everywhere: $O(N)$.

C++ short circuits `||`, so if the value is found on the left, the right subtree is never visited. That is a free optimisation, but the worst case is unchanged.

If the tree is a **BST**, you can do much better by using the ordering:

```
bool searchBST(TreeNode* root, int target) {
    if (root == NULL) return false;
    if (root->val == target) return true;
    if (target < root->val) return searchBST(root->left, target);
    return searchBST(root->right, target);
}
```

Now only one side is explored, so the cost is $O(H)$: $O(\log N)$ on a balanced BST. This is the payoff for the extra ordering promise.

Same tree

LeetCode 100. Two trees are identical if they have the same shape and the same values in the same places.

New idea: the function takes **two** pointers and walks both trees in step.

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == NULL && q == NULL) return true;    // both ended together
    if (p == NULL || q == NULL) return false;   // one ended first, shapes differ
    if (p->val != q->val) return false;

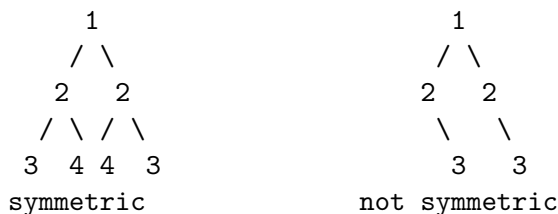
    return isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
```

The order of the first two lines matters. `&&` first, then `||`. If you write the `||` check first, you reject two empty trees, which are in fact equal.

Time $O(N)$ where N is the size of the smaller tree, since we stop at the first mismatch. **Space** $O(H)$.

Symmetric tree

LeetCode 101. Is the tree a mirror image of itself?



The mistake almost everyone makes first is to compare `root->left` with `root->right` using `isSameTree`. That checks whether the two halves are **copies**, not **mirrors**. In the symmetric picture above, the left half is $2(3, 4)$ and the right half is $2(4, 3)$. They are not the same tree.

The fix: write a helper that checks whether two trees are mirrors of each other, by pairing the left of one with the right of the other.

```

bool isMirror(TreeNode* a, TreeNode* b) {
    if (a == NULL && b == NULL) return true;
    if (a == NULL || b == NULL) return false;

    return a->val == b->val &&
           isMirror(a->left, b->right) &&    // outer pair
           isMirror(a->right, b->left);      // inner pair
}

bool isSymmetric(TreeNode* root) {
    if (root == NULL) return true;
    return isMirror(root->left, root->right);
}

```

Compare with `isSameTree`: the only difference is the crossing in the last two lines. Everything else is identical. Write both functions side by side once and the difference will stick.

Time $O(N)$, space $O(H)$.

The pattern behind all of this

Put the functions next to each other:

```

count      : if NULL -> 0          ; 1 + L + R
sum        : if NULL -> 0          ; val + L + R
height     : if NULL -> 0          ; 1 + max(L, R)
max value  : if NULL -> INT_MIN    ; max(val, L, R)
search     : if NULL -> false     ; (val == t) || L || R
leaves     : if NULL -> 0          ; leaf ? 1 : L + R

```

Same shape every time:

```
answer = combine(left subtree answer, right subtree answer, current node)
```

When you meet a new tree problem, do not ask “what algorithm is this”. Ask:

1. What single value would I like each subtree to hand me?
2. What is that value for an empty tree?
3. How do I merge two of them with my own node?

If you can answer those three, the code writes itself in five lines. If you cannot answer question 1 with a single number, you probably need to return a small bundle of values instead, and that is exactly what Part 6 is about.

Common mistakes

- **Missing base case.** Always the first line.
- **|| instead of && in the leaf check.** A leaf has both children NULL.
- **Assuming a missing child creates a leaf.** This is the minimum depth trap.
- **Wrong identity for the empty case.** Returning 0 for a max query breaks on all-negative trees.

- **Using `isSameTree` for symmetry.** Mirror is not the same as equal.
- **Recomputing.** Calling `maxDepth(root->left)` twice in one function doubles the work for no reason. Store it in a variable.
- **Global variables left dirty between test cases.** If you use a member variable such as `int best`, reset it at the start of the public function. On LeetCode the same object is reused across test cases and this causes bugs that only show up on the second case.

Check yourself

1. What should a height function return for NULL if you are counting edges, and why?
2. Why is the naive minimum depth solution wrong, and which tree exposes it?
3. `countNodes` and `sumOfNodes` differ by one token. Which one, and what does that tell you about the template?
4. What is the correct base value for a function that finds the minimum value in a tree?
5. Explain in one sentence why `isSymmetric` needs a two argument helper.
6. Search in a general binary tree is $O(N)$ but in a BST it is $O(H)$. What extra promise buys you that?
7. For a function that returns `true` only if every node value is even, what should the NULL case return?